

Technical Documentation: CrowdSense – Smart Crowd Detection & Management System

1. Project Overview

Objective and Purpose

CrowdSense is an edge-computing, IoT, and AI-driven system designed to actively detect, manage, and prevent crowd-related disasters, specifically stampedes, in real-time. The core purpose is to transition security measures from passive recording to active, automated threat prevention.

Problem Statement and Real-World Relevance

Traditional CCTV systems are strictly passive—they record events as they unfold but require constant human monitoring to identify danger. In high-density environments, human operators often notice overcrowding only after a critical incident has begun. There is a critical need for an automated system that can autonomously detect hazardous density and alert authorities proactively.

Key Features and Innovations

- **Edge-Processed AI:** Eliminates cloud-processing latency by running advanced YOLO-based object detection locally on IoT hardware.
- **Intelligent Density Clustering:** Mathematically calculates the physical distance between individuals to identify dangerous "clusters."
- **Sustained Threat Stopwatch:** A 3.5-second buffer algorithm that prevents false alarms caused by temporary camera obstructions.
- **Multi-Channel Alerting:** Automatically triggers physical onsite alarms, cloud-based mobile notifications, SMS, and automated voice calls.

2. Team Presentation Strategy & Role Mapping

This project is highly modular. During the presentation, the breakdown of system components and explanations will be handled as follows:

Flutter Application

- **Yash:** Will explain the **UI/UX Design, Navigation, and User Flow**. This includes the dark-mode aesthetic for high-stress environments, the single-page dashboard architecture, and how a security admin navigates from live feeds to incident logs.
- **Saheel:** Will explain the **API Calls, Data Display, and User Interaction**. This covers the real-time Firebase listeners, rendering dynamic density graphs, and the native "swipe-to-confirm" UI latch mechanism.

IoT Module

- **Saishwar:** Will explain the **Sensors, Data Collection, and Output Generation**. This includes detailing the hardware setup (Raspberry Pi 5, 1080p WebCam) and how the physical warning signals (sirens and LEDs) are triggered instantaneously on-site.

Backend Logic

- **Yash:** Will explain the **Server Flow, APIs, and Request-Response**. This covers the localized Flask streaming server, how video is broadcasted over the network, and the API design for external triggers (Twilio).
- **Saheel:** Will explain the **Database, Data Handling, and Logic**. This details the Firebase schema, the asynchronous upload queues for Cloudinary evidence, and how the system prevents data bottlenecks.

AI Model & Overall Process

- **Mevin & Chris Dias:** Will explain the **Model Working (Prediction/Decision) and Complete Flow**. This covers the YOLOv11 implementation, centroid distance math, the "Ghost Box" performance optimization, and the entire step-by-step pipeline from Input → Processing → Output.
- **Saishwar:** Will explain the **Integration with IoT & Backend**. This details the multi-threading architecture that allows the AI, network uploads, and hardware GPIO pins to run simultaneously without freezing the system.

3. System Architecture

High-Level Architecture

The architecture follows a distributed Edge-to-Cloud-to-Mobile topology.

1. **Edge/IoT Layer:** A high-definition camera feeds live video to the processing unit (Raspberry Pi).
 2. **AI/Processing Layer:** The AI model analyzes frames, calculates density metrics, and evaluates threat levels.
 3. **Cloud/Database Layer:** Data is synchronized instantly to a Firebase Realtime Database, while evidence images are securely pushed to a Cloudinary storage bucket.
 4. **Application Layer:** A Flutter mobile application listens to the cloud database, rendering live metrics and video feeds for the end-user.
-

4. Flutter Application Details

UI/UX Design Principles & Navigation

The application utilizes a dark-mode aesthetic to reduce glare and highlight critical color-coded alerts (Green for Safe, Orange for Warning, Red for Danger). The navigation follows a single-page dashboard architecture, ensuring immediate access to the live feed, current metrics, and historical logs.

API Integration, Data Display & User Interaction

- **Data Fetching:** Utilizes data streams to instantly reflect changes in crowd count and risk status with zero required page refreshes.
- **User Interaction (The Latch):** When a threat is detected, the UI deploys a "Latch" mechanism. The alert locks onto the screen, ignoring temporary fluctuations in the crowd, and requires deliberate human interaction.
- **Actionable UI:** Implements a native swipe-to-confirm interface. The admin must deliberately slide a toggle to mark an event as a "Confirmed Stampede" or a "False Alarm."

5. Backend System Details

Server Architecture & Workflow

The backend operates entirely on the edge device to ensure maximum privacy and minimum latency. It features a localized Flask streaming server that takes the heavily processed AI frames and broadcasts them as an MJPEG stream over the local network.

Database & Data Processing Logic

The backend utilizes asynchronous worker queues to handle heavy network requests. When the AI registers a sustained threat, the logic packages the data and pushes it to an upload queue, ensuring that pushing logs to Firebase and processing Twilio API requests do not interrupt the real-time camera feed.

6. Database & Data Handling

Database Structure and Schema

CrowdSense utilizes Firebase Realtime Database organized into geographical nodes (e.g., `zone_A`).

- **Live Metrics:** Stores volatile data like `live_count`, `dense_count`, and `status`.
- **Hardware Health:** Tracks `pi_is_online` and storage capacities.
- **History Logs:** A structured sub-node where each incident is recorded with a unique timestamp ID, description, verification type, and the secure URL of the evidence image.

Real-Time Synchronization

Data is kept lightweight to ensure sub-second sync speeds. Rather than uploading heavy video to the database, the system uploads a single snapshot to Cloudinary and only stores the secure URL text string in Firebase.

7. IoT Module (Raspberry Pi) Details

Hardware Components and Collection

The system is built on a Raspberry Pi 5 equipped with an active cooler. Video data is collected via a direct USB connection to a 1080p Kreo WebCam running at 60 FPS.

Signal/Output Generation & Backend Communication

Using the `gpiozero` library, the Pi directly controls localized hardware. When a critical threat is detected, the IoT module triggers an active buzzer and a high-intensity red LED. Communication with the backend is managed locally within the Python environment.

8. AI Model Details

Model Used and Input Processing

The system utilizes Ultralytics YOLOv11. High-definition frames are mathematically resized (e.g., `imgsz=640` or `768`) to balance detection distance with speed. The Intersection Over Union (IoU) parameter is strictly tuned (`iou=0.90`) to detect overlapping individuals.

Detection, Prediction, and Decision-Making Logic

The AI calculates the centroid of every detected human and executes a distance matrix algorithm. If multiple individuals fall within a predefined "Danger Radius" (e.g., 180 pixels), they are flagged as a "Dense Cluster." Decisions are based on two thresholds: maximum overall headcount OR maximum dense cluster count.

Performance Optimization (Ghost Box Technique)

To maintain high frame rates on a microcomputer, the AI only processes 1 out of every 10 frames (achieving 6 AI FPS). For the skipped frames, the algorithm projects the bounding boxes from the last known frame, creating the visual illusion of a 60 FPS feed.

9. End-to-End Workflow

1. **Input:** The USB camera captures a raw video frame.
2. **AI Processing:** The YOLO model generates bounding boxes. Centroids are mapped, and cluster density is calculated.
3. **Backend Decision:** The system checks thresholds. If exceeded, a real-time stopwatch begins.
4. **IoT Response:** If the threat is sustained for exactly 3.5 seconds, the Pi activates physical sirens and LEDs.
5. **User Notification (Cloud Sync):** The system captures a frame, uploads it to Cloudinary, logs data to Firebase, and triggers Twilio to execute a voice call and SMS.

6. **Mobile App Display:** The Flutter app detects the state change, locks into "High Alert," updates the risk graph, and waits for the admin to swipe a confirmation.

10. Integration

The system relies on seamless multi-threaded integration inside the Python environment:

1. **Capture Thread:** Pulls raw frames from the camera.
2. **AI Thread:** Analyzes frames and makes math-based threat decisions.
3. **Network Thread:** Handles Firebase Admin SDK updates, Cloudinary uploads, and Twilio REST API calls.

This decoupled architecture ensures that a slow internet connection to the cloud never causes the local IoT alarms or the AI detection to lag.

11. Challenges & Solutions

- **Hardware Overheating & Lag:** *Challenge:* Processing 60 FPS video crashed the Pi. *Solution:* Implemented the "Ghost Box" algorithm, reducing AI workload by 90% while maintaining visual smoothness.
- **UI Flickering:** *Challenge:* Minor crowd movements caused the mobile app buttons to flash rapidly. *Solution:* Implemented a software "Latch" that forces the alert to stay on screen until human intervention occurs.
- **False Positives:** *Challenge:* People briefly walking past the camera triggered alarms. *Solution:* Replaced frame-counting with a strict real-time stopwatch (`time.time()`), requiring 3.5 seconds of sustained danger before alerting.

12. Conclusion

Impact of the System

CrowdSense proves that enterprise-grade, proactive crowd management can be achieved using low-cost edge computing. By shifting the paradigm from passive observation to active intervention, the system can significantly reduce the response time of authorities.

Scalability and Future Improvements

The architecture is inherently scalable. Because the heavy AI processing is done on the edge device, adding more cameras simply requires adding more Pi units, without increasing cloud processing costs. Future improvements could include multi-camera tracking and predictive heat-mapping based on historical data.

Backend Code Summarized:

1. The Setup & API Keys (Lines 1-46)

- **The Imports:** You are loading the system's toolkit. This includes heavy computer vision libraries (`cv2`, `YOLO`), cloud SDKs (`firebase_admin`, `cloudinary`, `twilio`), and hardware management tools (`gpiozero`, `psutil`, `threading`).
- **The Keys:** This acts as the secure vault for the script. It holds the cryptographic credentials required for the Raspberry Pi to authenticate with the Firebase realtime database, push media to the Cloudinary storage bucket, and authorize the Twilio telephony API to make calls.
- **The Baseline Limits:** These are the default, hardcoded thresholds for the AI. It establishes that 30 people is a "Warning," 50 people is "Critical," and any two people standing within 180 pixels of each other constitute a "Density Risk." These serve as the foundational logic until the mobile app overrides them.

2. Hardware & Cloud Initialization (Lines 48-84)

- **Booting Up:** The script executes a digital handshake with Firebase and Cloudinary to ensure the connection is live. It also initializes the Twilio client so the dialer is pre-warmed and ready to fire without delay.
- **The AI Brain:** It loads `yolo11n.pt` into the Pi's RAM. Using the "nano" version of YOLO ensures the neural network runs as efficiently as possible on edge hardware.
- **The Camera Constraints:** It taps into the USB webcam (`CAMERA_INDEX = 0`). More importantly, it forces the hardware to strictly push a 1920x1080 resolution at 60 Frames Per Second (FPS). This ensures the AI gets a crystal-clear feed, even if people are far away.
- **The Hardware Alarms:** It maps the Pi's physical GPIO pins to the Green LED, Red LED, and the physical Buzzer. This creates the bridge between software logic and physical electrical outputs to warn the crowd.

3. The App Listeners & Shutdown Safety (Lines 86-121)

- **Listening to the App:** The `on_firebase_status_change` and `on_settings_change` functions are event-driven listeners. Instead of the Pi constantly asking the cloud if anything changed, Firebase instantly pushes updates to the Pi. If the admin adjusts the limits or clicks "False Alarm" on the Flutter app, the Pi's internal rules update in milliseconds without restarting the script.
- **The Graceful Death (`shutdown_handler`):** Unplugging a database-connected device can corrupt data. If the script is killed, this function catches the termination signal. It immediately turns off the physical sirens (so they don't get stuck screaming),

gives the Pi a 1-second grace period to tell Firebase `"pi_is_online": False`, and then cleanly exits.

4. The Upload Worker Thread (Lines 123-176)

- **The Bottleneck Problem:** Uploading an image to the cloud or waiting for a Twilio API response takes a few seconds. If the main video script had to wait for those internet tasks to finish, the live CCTV feed would freeze during an emergency.
- **The Asynchronous Solution:** You built a producer-consumer `upload_queue`. This acts as a background worker. When the AI detects a stampede, it tosses the alert data into this queue. The worker silently catches it and handles the slow Cloudinary uploads, Firebase updates, and Twilio dispatches in the background, keeping the camera feed buttery smooth.

5. The AI Processing Thread (Lines 178-362)

- **The Ghost Box Optimization:** Processing heavy AI math on 60 frames every second would melt a Raspberry Pi. Instead, the script only feeds 1 out of every 10 frames to YOLO. For the 9 skipped frames, it simply projects the bounding boxes from the last calculation. This creates the illusion of 60 FPS video while the AI comfortably runs at 6 FPS.
- **High-Fidelity Detection:** It scans the frames using a strict `iou=0.90` (Intersection Over Union). This ensures that even if people are overlapping in a tight crowd, the AI recognizes them as separate individuals instead of merging them into one box.
- **The Density Math (Centroids):** It finds the exact mathematical center point (centroid) of every detected person. Using the Pythagorean theorem (`math.hypot`), it measures the pixel distance between every single centroid. If people are crammed within 180 pixels of each other, they are flagged as a "Dense Cluster" and their bounding boxes dynamically turn Red.
- **The 3.5-Second Stopwatch:** The AI uses a non-blocking `time.time()` stopwatch. If the crowd exceeds the critical limits, the timer starts. The threat must be sustained for exactly 3.5 seconds. If the crowd clears out in 2 seconds, the timer resets, filtering out false alarms.
- **The Incident Trigger:** The exact moment the timer hits 3.5 seconds, the logic executes. It snaps a JPEG evidence photo, triggers the physical GPIO buzzer, and throws the alert payload into the upload queue to wake up the admin.
- **The Heartbeat Sync:** Every 0.5 seconds, it pings Firebase with the live headcount, cluster count, and the Pi's remaining SD card storage so the Flutter app's dashboard and charts stay perfectly updated.

6. The Camera Thread & Flask Server (Lines 364-386)

- **The Camera Thread:** The `capture_frames()` function exists for one sole purpose: to constantly pull raw image frames off the USB bus and drop them into a queue for the AI to pick up. Isolating this prevents hardware I/O bottlenecks.

- **The Local Web Server:** The `generate_feed` function encodes the AI-annotated frames into a continuous stream of JPEGs (MJPEG format). The Flask web framework then hosts this live video stream on port 5000 over the local network, serving as the direct feed for the Flutter app.

7. The Engine Starter (Lines 388-393)

- **The Orchestrator:** This block executes when the script runs. It initializes the three core systems (Uploads, AI Processing, Camera Capture) as parallel "daemon" threads. This means they run simultaneously, maximizing the Pi's multi-core processor, and then successfully boots the Flask server to bring the entire CrowdSense system online.

Software Requirements & Technical Specification

Project: CrowdSense Mobile Application

Platform: Flutter / Dart (Cross-Platform Mobile)

Lead Engineers: Yash (Frontend & Navigation Architect) & Saheel (Backend Integration & State Logic)

Version: 3

1. System Overview

The CrowdSense mobile application is the primary command-and-control interface for the CrowdSense hardware ecosystem. Built using the Flutter framework, it provides real-time telemetry, low-latency video streaming, and remote hardware control over edge-computed AI nodes (Raspberry Pi 5). The application is engineered for high-availability security environments, utilizing WebSockets for zero-delay data synchronization and asynchronous rendering to prevent UI thread blocking.

2. Application Architecture

The codebase strictly adheres to a modular, decoupled architecture to ensure separation of concerns between business logic, external services, and user interfaces.

Directory Structure

- **lib/components/**: Contains pure, stateless UI widgets. These are highly reusable design system elements independent of backend logic.
- **lib/pages/**: Contains stateful, screen-level widgets. Subdivided into **auth/** for unauthenticated flows and root-level files for authenticated dashboard views.
- **lib/services/**: Contains singleton classes and wrappers for external APIs (Firebase, Notifications, Local Storage).

3. Frontend UI/UX & Navigation Engineering (Lead: Yash)

The frontend architecture was designed with a focus on cognitive load reduction. In high-stress scenarios (e.g., an active stampede warning), the UI must instantly convey critical information without requiring deep navigation.

3.1 Design System & Theming

- **High-Contrast Dark Mode:** The app utilizes a deep dark color palette (Base: `Color(0xFF121212)`, Surface: `Colors.grey[900]`). This reduces eye fatigue in low-light control rooms and allows colored alerts to establish visual hierarchy.
- **Traffic-Light Status Indicators:** Risk levels are mapped to strict semantic colors:
 - Safe / Normal = **Green**
 - High Density = **Orange / Amber**
 - Critical Risk / Danger = **Red**

3.2 Navigation Architecture

- **Initialization (`splash_page.dart`):** Handles asynchronous bootstrap tasks (checking secure storage for existing auth tokens, initializing Firebase) before routing.
- **Persistent Bottom Navigation (`main_nav.dart`):** Implements an `IndexedStack` combined with a `BottomNavigationBar`. This ensures that when a user switches from the `live_feed_page` to the `map_page`, the video stream and WebSocket connections are not destroyed and rebuilt. The state is preserved in memory.

3.3 Custom Component Library

To maintain a DRY (Don't Repeat Yourself) codebase, core elements were abstracted:

- **`my_button.dart` & `my_textfield.dart`:** Standardized input and action widgets ensuring consistent padding, typography, and touch-target sizes.
- **`stat_card.dart`:** A dynamic container that accepts `currentCount` and `riskLevel` variables, automatically updating its border and icon colors based on the data payload.

3.4 Interactive Elements: The Swipe-to-Confirm UI

Standard `ElevatedButton` widgets are prone to accidental taps. For the critical "Kill Switch" functionality (disarming the physical Raspberry Pi sirens), the UI employs Flutter's `Dismissible` widget.

- **Implementation:** The user must intentionally drag a circular icon across a horizontal track. This mechanical friction ensures that "Confirm Stampede" or "False Alarm" actions are deliberate human decisions.

4. Backend Integration & State Management (Lead: Saheel)

The backend logic is heavily reliant on reactive programming. The app does not "ask" for data; it listens to an open data stream and mutates its state asynchronously.

4.1 Real-Time Telemetry (Firebase Streams)

- **Mechanism:** Inside `initState()`, the app binds a listener to the Firebase Realtime Database path (`crowd_monitor/zone_A`).

- **Execution:** `_dbRef.onValue.listen((event)` creates a persistent WebSocket. When the Raspberry Pi updates the crowd count, the event triggers instantly. The JSON payload is parsed, and `setState()` is called to rebuild only the necessary text nodes on the UI, resulting in sub-100ms updates.

4.2 Asynchronous Video Streaming

Streaming 1080p video directly through a database is impossible.

- **Solution:** The app uses the `flutter_mjpeg` library. It opens a direct HTTP connection to the Raspberry Pi's Flask server (`http://<PI_IP>:5000/video_feed`).
- **Error Handling:** The stream is wrapped in an asynchronous builder. If the Pi loses power, the `errorBuilder` catches the timeout and renders a "Camera Offline" UI instead of throwing a fatal exception that crashes the app.

4.3 Algorithmic State Control: "The Latch"

A critical challenge was UI flickering. Because the YOLO AI recalculates 6 times a second, the status could rapidly fluctuate between "CRITICAL RISK" (21 people) and "Normal" (19 people).

- **The Latch Logic:**

```
dart
if (status == "CRITICAL RISK") {
  isHighAlert = true;
}
```
- By explicitly omitting an `else { isHighAlert = false; }` statement inside the stream listener, the UI "latches" into emergency mode. Once triggered, the app ignores subsequent "Normal" signals from the Pi. The alert state can only be deactivated by the human operator utilizing the swipe actions.

4.4 Data Parsing & Visualization (`fl_chart`)

The app visualizes the `history` sub-node containing past incident logs.

1. **Ingestion:** The raw map from Firebase is converted into a `List<MapEntry>`.
2. **Sorting:** `entries.sort()` is applied using the Unix timestamp integers to ensure chronological ordering.
3. **Graph Plotting:** The application iterates over the sorted logs, converting the `people_count` integers into `FISpot(x, y)` coordinates, mapping them onto an interactive spline chart.

5. Critical Workflows & Event Resolution

5.1 The Remote Hardware Override (Kill Switch)

When the user swipes the action bar, the `_handleAlarm(bool isTrueAlarm)` function executes a multi-node atomic write to the cloud.

1. **Log Auditing:** It targets the specific `latestLogKey` in the `history` folder and updates its `type` from "Pending" to either "True" or "False".
2. **Hardware Reset:** It updates the root `zone_A` node, forcing `'status': 'Normal'` and toggling `'false_alarm_flag'`.
3. **IoT Response:** Because the Raspberry Pi is listening to this exact Firebase node, the cloud update forces the Python script to execute `set_alarm_outputs(False)`, immediately cutting power to the physical onsite buzzers and LEDs.

6. Services Layer Documentation

The `lib/services/` directory isolates core functionality from the widget tree.

- **`auth_service.dart`:** Wraps the `FirebaseAuth` instance. Handles user creation, login validation, and securely stores session tokens locally using `shared_preferences`.
 - **`notification_service.dart`:** Configures Firebase Cloud Messaging (FCM). Requests OS-level permissions and handles background message processing so the phone receives alerts even if the app is closed.
 - **`tflite_service.dart` (Experimental Module):** Built to explore offline, edge-based processing directly on the mobile device. This class wraps the TensorFlow Lite interpreter, allowing the mobile application to run localized object detection models if cloud connectivity fails.
-

7. Data Contract (Expected Firebase Schema)

The application state relies on the following structured JSON payload from the IoT module:

JSON

```
{
  "zone_A": {
    "live_count": 45,
    "dense_count": 3,
    "status": "CRITICAL RISK",
    "pi_is_online": true,
    "false_alarm_flag": false,
    "settings": {
      "warning_limit": 30,
      "critical_limit": 50
    },
    "history": {
      "-Nxyz123abc": {
        "timestamp": 1714321234567,
        "description": "Stampede Density Detected",
        "type": "Pending",
        "people_count": 45,
        "image_url": "https://res.cloudinary.com/..."
      }
    }
  }
}
```



CrowdSense: Complete System Workflow Architecture

Phase 1: Raw Data Ingestion (The Edge)

- **Hardware Capture:** The system begins at the physical venue with a 1080p Kreo WebCam. It captures raw video footage of the crowd at a high frame rate (60 FPS).
- **Local USB Transfer:** This raw, uncompressed video data is fed directly into the local edge computing node (Raspberry Pi 5) via USB. At this stage, the data is purely visual with no telemetry or intelligence applied.

Phase 2: AI Edge Processing (The Brain)

- **The "Ghost Box" Optimization:** To prevent thermal throttling on the microcomputer, the Python capture thread intentionally skips processing every single frame. It isolates 1 out of every 10 frames to feed into the YOLOv11 AI model.
- **Object Detection:** The YOLOv11 model mathematically resizes the frame and detects all human profiles, generating coordinate bounding boxes and confidence scores.
- **Density Matrix Calculation:** The system maps the exact center (centroid) of every detected human. It then executes a Pythagorean distance algorithm (`math.hypot`) across all centroids. If multiple individuals are packed tightly within a predefined pixel radius (e.g., 180 pixels), they are categorized as a "Dense Cluster."

Phase 3: Threat Verification (The Logic)

- **Threshold Monitoring:** The system constantly compares the live `current_total` and `current_dense_count` against the dynamic limits configured by the admin in the mobile app.
- **The 3.5-Second Stopwatch:** To filter out false positives (e.g., a large group swiftly walking past the lens), the system employs a strict time-buffer. If a threshold is breached, a background timer (`time.time()`) initiates.
- **The Trigger Point:** The crowd must sustain that critical threshold continuously for exactly 3.5 seconds. If the count drops even for a millisecond, the clock resets. If it hits 3.5 seconds, the system officially declares an active threat.

Phase 4: Local Alarms & Cloud Payload (The Nervous System)

- **IoT Hardware Activation:** Instantly, the Raspberry Pi utilizes the `gpiozero` library to send voltage to the physical GPIO pins, triggering an active siren and high-intensity red LEDs at the venue to initiate physical crowd dispersal.

- **Evidence Generation:** The camera thread takes a localized high-resolution `.jpg` snapshot of the exact frame that triggered the 3.5-second rule.
- **Asynchronous Queueing:** To prevent the live video feed from lagging, the system offloads all internet-based tasks to a background `upload_worker` queue.

Phase 5: External API Dispatch (The Broadcaster)

- **Cloudinary Storage:** The worker thread uploads the temporary evidence snapshot to a secure Cloudinary storage bucket and retrieves a permanent, secure image URL.
- **Twilio Dispatch:** The system packages the exact Google Maps coordinates of the specific zone and sends a POST request to Twilio. Twilio dispatches an emergency SMS and initiates a robotic Voice Call to the admin's device. A "call lock" is engaged so the system does not spam-call the admin repeatedly for the same incident.
- **Firebase Synchronization:** The worker thread pushes the final JSON payload (Headcount, Cluster Count, Timestamp, and Cloudinary URL) into the Firebase Realtime Database and updates the global zone status to `"CRITICAL RISK"`.

Phase 6: Mobile Application Mutation (The Control Room)

- **WebSocket Listener:** The Flutter mobile application, sitting idly in the administrator's pocket, receives the state change from Firebase in less than 100 milliseconds via an open WebSocket stream (`onValue.listen`).
- **UI Latch Execution:** The Dart logic detects the `"CRITICAL RISK"` string. It forces the `isHighAlert` boolean to true, locking the UI into emergency mode.
- **Dynamic Rendering:** The application instantly drops the dark mode into a red "Danger" theme, updates the `fl_chart` density graphs, and displays the swipe-to-action sliders.
- **Live Assessment:** The admin opens the application and views the evidence photo in the "Past Logs" section, alongside the live MJPEG CCTV feed streaming directly from the Pi's Flask server.

Phase 7: Event Resolution & System Reset (The Feedback Loop)

- **Human Verification:** The security administrator evaluates the live video and evidence. They interact with the physical `Dismissible` widget, sliding the toggle to either "Confirm Stampede" or "False Alarm."
- **The Reverse Signal:** The Flutter application dispatches a patch request to Firebase, logging the human verification outcome and resetting the root zone status back to `"Normal"`.

- **Hardware Stand-Down:** The Raspberry Pi's own Firebase listener catches this "Normal" status update from the cloud. It executes a local override, instantly killing the power to the physical GPIO sirens, resetting the Twilio call lock, and resuming standard passive monitoring.